Object-Oriented Design Patterns

Info exercises

Object-Oriented Design Patterns

Object-Oriented Design Patterns

Design a Parking Lot

Design a Movie Ticket Booking System

Design a Vending Machine

Design a Cache

Design an Elevator System

Design a Stack Overflow

Design a Pastebin

Design a Blackjack Game

Design a Chess Game

Design an LFU System

Design a Hotel Reservation System

Design an Amazon Locker

Design a Tic Tac Toe Game

Design a Minesweeper Game

06

Design a Unix File Search System

In this chapter, we will explore the design of a Unix-like Search system. The goal is to design classes that represent abstractions of the underlying file system, such as files, directories, links, and filters. Along the way, we will learn how to create a clean and flexible structure that supports the standard interactions between these components, ensuring the search system is intuitive and scalable.

```
$ ls -l
$ find . -name *.mp4
$ find . -type f -size +100MB -modified -7d
$ find . -name "*.log" -modified -7d -size +10M -exec rm {} \;
$ find . -name "*.log" -modified -7d -size +10M -exec ls -l {} \;
$ find . -name "*.log" -modified -7d -size +10M -exec ls -l {} \; | sort -k5 -n
```

Unix-like search

Let's gather the specific requirements through a simulated interview session.

Requirements Gathering

Interviewer: Let's consider a simplified version of the Unix find command.

Interviewer: Your system should support filtering by file properties, such as name, type, size, and modification time.

Interviewer: The system should support compound filters, combining them with logical AND, OR, and NOT operations.

Interviewer: The system should also support actions, such as printing the path of a matching file or deleting it.

Interviewer: Finally, it should support traversing directories recursively, including the ability to follow symbolic links.

Requirements clarification

Here's an example of how a user would interact with the system:

Candidate: What attributes does the find command use to search for files?

Interviewer: It could search by file name, file type, filesize, and last modified time.

Candidate: Does it need to handle directories?

Interviewer: Yes, directories are a fundamental part of the system.

Candidate: What type of actions should the command support?

Interviewer: The core actions should be printing the matching path (the default action), and a "rm" action to delete the matched files.

Candidate: Can we combine multiple criteria, such as the same attributes?

Interviewer: Yes, it should support AND, OR, and NOT, as well as parentheses.

Candidate: I assume we're designing a system to search a directory and its sub-directories, returning files that match the given criteria.

Interviewer: Yes, that's a fair assumption.

Constructing concrete examples

With the requirements for our core Unix-like search system in place, let's take a look at concrete scenarios and how we would interact with the system. This will help us identify the key objects and their responsibilities.

- **Start with a Simple Search:** A user might begin by searching for all files in the current directory.
- **Scale Up to a Complex Search:** Then they might refine their criteria (e.g., files > 10 and size < 5000 and name = "*.txt") or add actions (e.g., find . -type f -print -exec rm {} \;).

Requirements

Based on the questions and answers, the following functional requirements can be identified:

- The search system can search for files based on attributes such as size, type, filename, and owner.
- The system should support compound filters (expressions) using logical AND, OR, and NOT operations, as well as parentheses for grouping.
- The system should support actions, such as printing the path of a matching file (the default action) and deleting matched files.
- The search system can traverse directories and sub-directories as well as files.
- The search system can apply search criteria to directories as well as files.

Below are the non-functional requirements:

- Extensibility: Support adding new attributes (e.g., modification time) and new actions (e.g., logging) without modifying core components.
- Performance: Efficiently traverse large directory trees, minimizing memory usage and avoiding unnecessary file system calls.
- Maintainability: Support adding new filters (e.g., modification time) and actions (e.g., logging) without modifying core components.
- Robustness: Handle errors (e.g., permission denied) gracefully, ensuring the search continues where possible.

Identify Core Objects

Now that we've seen how the search works, it's time to design a system that can handle them. Let's break down our core requirements. We need to find, filter, and act on files. This suggests three fundamental concepts: the file itself, the criteria we use to find it, and the action we take on it.

- **Filesystem:** We need a way to represent files and directories, ensuring we can abstract the underlying file system. This will allow us to perform searches on different file systems (e.g., local, remote, or even in-memory) without changing the core logic. We'll define a common interface for all file-like objects.
- **Filters:** We need to represent the search criteria, such as name, size, or modification time. These filters will be used to evaluate whether a file matches the user's query. We'll need a way to combine multiple filters using logical AND, OR, and NOT operations.
- **Actions:** We need to represent the actions that can be performed on the files, such as printing the path or deleting a file. These actions will be triggered when a file matches the search criteria.

Design choice: We define Filesystem as an abstract or protocol. Real-world implementations (e.g., LocalFileSystem, InMemoryFileSystem) will inherit from this. This ensures that only well, predefined attributes are used when evaluating files, preventing runtime errors and ensuring data integrity. The Filter class will have methods like matches(File file) and getFailureReason(File file), which will be used by the search engine. This design adheres to the Open/Closed Principle, allowing us to add new filter types (e.g., SizeFilter, ModificationTimeFilter) without modifying the core search logic.

FileSearch

The FileSearch class is responsible for traversing the file system tree in pre-order. This, using a FileSearchVisitor object, to select files that match the search criteria. The traversal should be iterative to avoid stack overflow errors with deep directory trees. We'll use a queue or stack to manage the traversal. The FileSearch class will also be responsible for applying the specified actions to the matched files.

FileSearchCriteria

The FileSearchCriteria class acts as a data transfer object, encapsulating the search criteria. It will contain a list of Filter objects and a boolean flag indicating whether to perform a recursive search. This ensures that all the search parameters are passed to the FileSearch class in a single, well-defined object.

Predicate and SimplePredicate

The Predicate interface defines a contract for functions that evaluate whether a file should be included in the search results. The SimplePredicate class provides a concrete implementation of this interface, wrapping a single function that takes a File as input and returns the result. This allows us to encapsulate the logic for checking file attributes (e.g., size, name, modification time) within a reusable and testable unit.

ComparisonOperator

The ComparisonOperator interface defines a contract for comparing a field attribute value (like size or name) to an expected value. This allows us to decouple the comparison logic from the specific attribute being evaluated. We can then have different implementations that define how to compare different data types (e.g., integers, strings, dates). This makes it easy to add new comparison operators (e.g., greater than, less than, contains) without modifying the core comparison logic.

Comparison operators: Interface and concrete classes

Alternative approach: We could represent operations with strings such as "equals" or "<". This simplifies initial implementation but introduces potential runtime errors if we encounter unexpected strings. Using a dedicated interface provides more flexibility and more robust error handling (e.g., throwing a custom exception if an invalid operation is used).

Another option is to use enums like EQUALS or GREATER_THAN to represent comparison operators. This makes the code safer and more readable than using strings. However, it still limits extensibility slightly, as adding new operators would require modifying the existing enum. Overall, using an interface is the most flexible approach, as it allows us to change the underlying implementation without impacting the rest of the system. For this exercise, we'll use a simple class for the new operation without worrying about scaling.

Composite Predicate

To handle complex search criteria with multiple comparisons implementations in place, we can simply nest them against single conditions. For example, to find files larger than 10 MB and modified within the last 7 days, combining multiple conditions (size and modification time), we'd create two SizeFilter and DateFilter objects, enabling us to build complex predicates from simpler ones.

To represent more complex logic, such as "size > 10 and name = "*.log" or modification time < 7 days", we'll introduce a CompositePredicate. This class will combine multiple predicates using logical operators (AND, OR, NOT). This allows us to create arbitrarily complex search criteria by combining simple predicates in a tree-like structure.

Composite Predicate Interface and concrete classes

With all classes defined, let's review how they fit together in the complete class diagram.

Complete Class Diagram

Having built our classes, there's now the hierarchical file structure to the intricate predicate logic, we're ready to see the complete picture of our Unix-like search system. The diagram below illustrates the relationships between the various components we've designed.

File
+ boolean isDirectory()
+ boolean matchesFilter(Filter f)
+ void accept(FileVisitor v)
+ Collection<File> getChildren()
+ boolean delete()
LocalFileSystem: optional
+ File getFile(String path)
InMemoryFileSystem: optional
+ File getFile(String path)
+ void addFile(File f, String path)
Filter
+ boolean matches(File f)
+ boolean matchesFilter(File f)
+ boolean matchesFilter(File f)
+ boolean matchesFilter

```
    // The predicate that defines what makes a file match
    private final Predicate predicate;

    // Constructor that takes a predicate defining the criteria
    public FileSearchCriteria(final Predicate predicate) {
        this.predicate = predicate;
    }

    // Checks if the given file matches the search criteria
    public boolean isMatch(final File inputFile) {
        return predicate.isMatch(inputFile);
    }
}
```

This design centers on the isMatch method, which invokes the Predicate's logic to assess a File and returns true for matches, such as "size > 10" with SimplePredicate or "size > 10 and owner = 'bob'" with CompositePredicate.

Implementation choice: We chose to wrap the Predicate in FileSearchCriteria to keep FileSearch focused solely on filesystem traversal and isolate condition evaluation to a separate, reusable layer.

Alternate implementation: Instead of using FileSearchCriteria, FileSearch could directly invoke the Predicate's logic to evaluate files. However, this would tightly couple traversal and filtering logic, making it harder to modify or extend conditions independently.

FileSearch

The FileSearch class orchestrates the search across the filesystem, starting from a root File and leveraging FileSearchCriteria to identify matching files through recursive traversal. It employs a stack-based approach to explore directories efficiently. Here's the implementation:

```
// Main class responsible for performing file system searches
public class FileSearch {
    // Performs a recursive search through the file system starting from root
    // Returns a list of files that match the given criteria
    public List<File> search(final File root, final FileSearchCriteria criteria) {
        // List to store matching files
        final List<File> result = new ArrayList<>();
        // Stack to handle recursive traversal without actual recursion
        final ArrayDeque<File> recursionStack = new ArrayDeque<>();
        // Start with the root directory
        recursionStack.add(root);
        // Continue until we've processed all files
        while (!recursionStack.isEmpty()) {
            // Get the next file to process
            File next = recursionStack.pop();
            // Check if the file matches our criteria
            if (criteria.isMatch(next)) {
                result.add(next);
            }
            // Add all directory entries to the stack for processing
            for (File entry : next.getEntries()) {
                recursionStack.push(entry);
            }
        }
        return result;
    }
}
```

This search method begins by initializing a result list and an ArrayDeque stack with the root File. It then processes each file by popping it from the stack, evaluating it with the criteria.isMatch, and adding matches to the result. For directories, it pushes all entries onto the stack, ensuring every file is visited in a depth-first manner. The returned List<File> contains all matches, mutable for flexibility in downstream use, by the client code.

Implementation choice: We opted for a stack over recursive calls to prevent stack overflow in deep filesystems, balancing efficiency with robustness. Alternatively, a recursive method could work for smaller structures, but it risks failure on large or deeply nested directories.

Deep Dive Topic

Now that the basic design is complete, the interviewer might ask you some deep dive questions. Let's check out some of these.

File search test

After implementing our classes, it's a good idea to verify the end-to-end logic. The UNIX file search problem is abstract, so we create a test case to demonstrate how it works.

Here's a test case for the condition "non-directories owned by 'ge.*'":

```
public class FileSearchTest {
    @Test
    public void testFileSearch() {
        // Create a root directory and two files with different owners
        final File root = new File(true, 0, "adam", "root");
        final File a = new File(false, 2000, "adam", "a");
        final File b = new File(false, 3000, "george", "b");

        // Add files to the root directory
        root.addEntry(a);
        root.addEntry(b);

        // Search criteria: Find non-directory files owned by users matching "ge.*"
        final FileSearchCriteria criteria =
                new FileSearchCriteria(
                        new AndPredicate(
                                List.of(
                                        new SimplePredicate<>(
                                                FileAttribute.IS_DIRECTORY,
                                                new EqualsOperator<>(),
                                                false),
                                        new SimplePredicate<>(
                                                FileAttribute.OWNER,
                                                new RegexMatchOperator<>(),
                                                "ge.*"))));

        // Execute the search and get results
        final FileSearch fileSearch = new FileSearch();
        final List<File> result = fileSearch.search(root, criteria);

        // Verify that only one file matches the criteria
        assertEquals(1, result.size());
        // Verify that the matching file is "b"
        assertEquals("b", result.get(0).getFilename());
    }
}
```

Wrap Up

With the UNIX file search system fully implemented and tested, it's time to step back and consider what we've achieved. This chapter began by gathering requirements through a structured dialogue, then progressed to defining core objects, crafting their class structure, and coding the essential components.

The system's maintainability and extensibility are ensured by the clear division of responsibilities among the classes: File, FileSearch, FileSearchCriteria, and Predicate, which respectively represent files, traverse directories, evaluate conditions, and define match logic. Our choices, such as separating FileSearch from FileSearchCriteria and using generics in ComparisonOperator, improve scalability and maintain type safety throughout processes. We could have merged FileSearchCriteria with Predicate into one class for a tighter initial design, but this would blur their distinct roles, making it harder to update or swap condition logic without affecting traversal.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Further Reading: Composite Design Pattern

This section gives a quick overview of the design patterns used in this chapter. It's helpful if you're new to these patterns or need a refresher to better understand the design choices.

Composite design pattern

Composite is a structural pattern that lets you organize objects into tree structures and then handle these structures as if they were individual objects.

Problem

Imagine you have two types of objects: Files and Folders. A Folder can contain several Files as well as many smaller Folders and so on. Say you decide to create a search system that uses these classes. Searches could involve simple Files on their own, as well as Folders packed with Files, and other Folders. How would you find all items matching a specific condition, like "size > 10," across this structure?

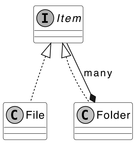
Solution

The Composite pattern suggests that we work with Files and Folders through a common interface that declares a method for checking conditions.

Here's how this method works:

- **For a File**: It checks if the File matches the condition, like "size > 10," and gives a yes or no answer.
- **For a Folder**:
  - It examines each item within, testing whether it meets the condition.
  - It recursively applies this process to any nested folders, traversing the entire tree until all items are evaluated.
  - It can also enforce its constraint, such as "not a directory," to refine the result.

Here's a simple diagram showing the Composite pattern for Files and Folders:



Item interface and concrete classes

Item is the common interface that both File and Folder use. The advantage is that we can treat all objects, File or nested Folder, the same through the common interface, letting them pass the check down the tree without knowing their types.

When to use

The Composite design pattern is useful in scenarios where:

- You need to build a tree-like object structure.
- When you want the client code to handle both simple and complex elements uniformly.

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hi@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Ask Alex

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.